



Hadil Ben Abdallah
@hadil-ben-abdallah

40 Python Questions Recruiters Ask in 2025



Introduction

Python is hotter than ever 🔥, and in 2025, companies are raising the bar even higher for devs who want to stand out. Whether you're prepping for your next big interview 🧑💻 or just want to test your Python muscle, you're in the right place!

Here, I've gathered **40 must-know Python interview questions**, ranging from beginner basics to advanced challenges. Each question includes a brief note on **why it matters**, followed by a **clear answer**. Think you've got what it takes? Let's find out!



- ✓ **Sharpen your skills**
- ✓ **Ace your next interview**
- ✓ **Become the Python pro companies are hunting for in 2025**

Let's dive right in! 🏊♂️

How the 40 Questions Are Structured

To make it more challenging, the questions are grouped into 4 levels, each one getting a little tougher:

Q1–10 → 🧐 **Beginner Level (Warm-Up Round)**

Get comfy with Python basics: syntax, data types, and core concepts.

Q11–20 → 🧑 **Intermediate Level (It Gets Real)**

Time to flex: decorators, generators, exceptions, and memory management.

Q21–30 → 🧑 **Advanced Level (Boss Mode)**

Deep dive: multithreading, coroutines, metaclasses, and optimization tricks.

Q31–40 → 😬 **Expert Level (Only the Brave)**

The final boss: internals, descriptors, async magic, and system design.

Challenge yourself!

Start from the top or jump straight into the level that matches your skills.



1

What is Python and what are its key features?

Interviewers start here to confirm you know Python's identity and what makes it unique in today's ecosystem.

Answer:



Python is a high-level, interpreted language known for its **readable syntax**, **dynamic typing**, and **vast standard library**. It supports multiple paradigms (procedural, object-oriented, functional) and boasts automatic memory management and a large ecosystem of third-party packages.

2

How is Python interpreted? 🧵

Understanding Python's execution model helps explain performance characteristics and debugging approaches.

Answer:

Python code is first compiled to **bytecode**, which is then executed by the **CPython virtual machine**. This two-step process (source → bytecode → interpreter) makes Python portable but can introduce runtime overhead.



Hadil Ben Abdallah
@hadil-ben-abdallah

3

What are Python's data types? 📖

Data types form the backbone of any program, knowing them ensures you choose the best structure for each task.

Answer:

Common Python built-in types include **integers, floats, strings, booleans, lists, tuples, sets, and dictionaries.**

Additionally, Python supports **NoneType** and user-defined classes.



4

What is PEP8 and why is it important? 📏

PEP8 enforces **consistent code style**, which improves readability and maintainability in team environments.

Answer:

PEP8 is the **Python Enhancement Proposal** that defines style guidelines, indentation, naming conventions, line length, etc. Adhering to PEP8 makes code predictable and easier to review.



5

Explain the difference between a list and a tuple.

Lists and tuples are fundamental collections, knowing their trade-offs affects performance and mutability choices.

Answer:

A **list** is **mutable** (you can add, remove, or change items), while a **tuple** is **immutable** (fixed after creation). Tuples can be used as keys in dictionaries and are slightly faster due to immutability.



6

What is the use of self in Python classes? 🏷️

“self” is central to instance methods, misusing it leads to confusing bugs.

Answer:

“self” refers to the **instance** on which a method is called, allowing access to attributes and other methods. It must be the first parameter of instance methods but is passed automatically by Python.



7

How is memory managed in Python? 🧠

Effective memory management prevents leaks and performance issues, especially in long-running applications.

Answer:

Python uses **reference counting** and a **generational garbage collector** to reclaim unreferenced objects. Cyclic references are cleaned up by the garbage collector's separate cycle-detection mechanism.



8

What is the difference between deep copy and shallow copy? 📄 VS 📄

Copy semantics can introduce subtle bugs if you don't know whether nested objects are shared or duplicated.

Answer:

A **shallow copy** creates a new container but references the same inner objects. A **deep copy** (via “`copy.deepcopy`”) recursively duplicates all nested objects, yielding a fully independent clone.



Hadil Ben Abdallah
@hadil-ben-abdallah

9

What are Python's built-in data structures? 🍵

Knowing your toolbox ensures you pick the most efficient structure for each problem.

Answer:

Python provides **list**, **tuple**, **set**, **frozenset**, **dict**, and **str** as core data structures. Each has different performance characteristics and use-cases based on mutability and ordering.



10

How do you manage packages in Python?

Package management keeps environments reproducible, a must in collaborative and production settings.

Answer:

You use **pip** (the Python package installer) to install and manage packages from the Python Package Index (PyPI). Virtual environments (“venv” or “virtualenv”) isolate dependencies per project. 



11

What are Python decorators and how are they used? 🎀

Decorators enable reusable, clean modifications to function or method behavior, critical for frameworks and logging.

Answer:

A decorator is a function that **wraps another function**, adding behavior before or after its execution. You apply it with “@decorator_name” above the function definition.



12

**What is list comprehension?
Give an example.** 

List comprehensions provide a concise, Pythonic way to build lists, improving readability and often performance.

Answer:

List comprehension uses a single expression to create a list:

```
1 squares = [x*x for x in range(10)]
```

This is equivalent to a for-loop but more compact.

13

Explain generators and “yield” in Python.

Generators allow you to handle large or infinite sequences lazily, saving memory.

Answer:

A generator function uses “yield” to produce a sequence of values one at a time. On each call to “next()”, execution resumes after the last “yield”, maintaining state between iterations. 



14

What are `*args` and `**kwargs`?

These constructs allow flexible function signatures, enabling wrappers, wrappers, and DSL-style APIs.

Answer:

“`*args`” captures extra positional arguments as a tuple, “`**kwargs`” captures extra keyword arguments as a dict. They let you accept arbitrary parameters.



15

How does exception handling work in Python?

Robust error handling is essential for building resilient applications.

Answer:

Use “try”/”except” blocks to catch exceptions, “else” for code that runs if no exception occurs, and “finally” for cleanup. You can raise custom exceptions with “raise”.



16

What is the difference between “is” and “==”? 🔍

Misunderstanding identity vs. equality can cause subtle, hard-to-trace bugs.

Answer:

“==” tests **value equality** (via “__eq__”), while “is” tests **object identity** (same memory address). Use “is” for singletons like “None”.



17

How do you manage memory leaks in Python? 🖌️

Even with GC, lingering references or large caches can bloat memory over time.

Answer:

Identify leaks with tools like “objgraph” or “tracemalloc”, remove unintended references, use **weak references**, and clear caches or large data structures when no longer needed.



18

What is a lambda function? 🚀

Lambdas let you define inline, anonymous functions, handy for short callbacks and functional APIs.

Answer:

A “lambda” is an anonymous function:



Code

```
1 square = lambda x: x*x
```

It's limited to a single expression and returns its result automatically.

19

What's the purpose of virtual environments in Python?

Isolated environments prevent dependency conflicts between projects.

Answer:

Virtual environments (via “venv” or “virtualenv”) create isolated Python installations per project, each with its own “site-packages”, so packages don’t collide.



Hadil Ben Abdallah
@hadil-ben-abdallah

20

Explain Python's Global Interpreter Lock (GIL).

The GIL impacts concurrency, knowing its limitations guides architecture and performance decisions.

Answer:

The GIL is a mutex that allows only one native thread to execute Python bytecode at a time in CPython. It simplifies memory management but limits CPU-bound multithreading, pushing developers toward multiprocessing or async.



21

How does multithreading work in Python despite the GIL? 🧵

Concurrency questions probe your ability to build responsive apps despite Python's threading model.

Answer:

Because of the GIL, Python threads excel at **I/O-bound tasks** (waiting on network/disk). For CPU-bound tasks, you use the “multiprocessing” module or offload work to C extensions that release the GIL.



Hadil Ben Abdallah
@hadil-ben-abdallah

22

What is monkey patching in Python? 🐒

Monkey patching lets you alter behavior at runtime, but can introduce maintenance nightmares if misused.

Answer:

Monkey patching dynamically modifies classes or modules at runtime, e.g.



Code

```
1 import module  
2 module.func = new_func
```

Use sparingly and document changes to avoid unpredictable behavior.

23

How are Python's memory management and garbage collection implemented? 🧹♻️

Deep understanding of GC internals helps you tune performance and avoid leaks.

Answer:

Python uses **reference counting** for immediate deallocation plus a **cyclic collector** (three generations) to detect and clean up reference cycles.



Hadil Ben Abdallah
@hadil-ben-abdallah

24

Explain context managers and the with statement.

Context managers ensure resources (files, locks) are properly acquired and released.

Answer:

A context manager defines “__enter__” and “__exit__”. The “with” statement calls these, ensuring cleanup: 

```
Code
1 with open('file.txt') as f:
2     data = f.read()
```

25

What is a metaclass in Python? 🧙

Metaclasses let you customize class creation, a powerful but advanced feature for frameworks.

Answer:

A metaclass is the “type of a class.” By subclassing “type”, you can override “__new__” or “__init__” to modify class attributes at creation time.



26

How do coroutines differ from threads? ⚙️

Coroutines enable high-concurrency I/O without threads, crucial in async frameworks.

Answer:

Coroutines use cooperative multitasking within a single thread, yielding control with “await”. Threads preemptively multitask, but in CPython share the GIL.



27

What is the difference between asynchronous programming and multithreading? ⚡

Choosing the right concurrency model impacts scalability and complexity.

Answer:

Async uses a single thread and event loop to interleave tasks on I/O waits. Multithreading uses OS threads to run tasks in parallel (but still limited by the GIL for CPU-bound code).



Hadil Ben Abdallah
@hadil-ben-abdallah

28

What are data classes (@dataclass) in Python 3.7+? 📁

Data classes reduce boilerplate for classes that mainly store data, improving readability.

Answer:

“@dataclass” automatically generates “__init__”, “__repr__”, “__eq__”, and other methods based on class annotations:

```
Code
1 from dataclasses import dataclass
2
3 @dataclass
4 class Point:
5     x: int
6     y: int
```

29

How would you optimize a slow Python script?



Performance tuning is a key skill for production-grade applications.

Answer:

Profile with “cProfile” or “line_profiler”, optimize hotspots by using built-ins, C extensions, or concurrency (async/multiprocessing), and reduce I/O or algorithmic complexity.



Hadil Ben Abdallah
@hadil-ben-abdallah

30

How does Python implement closures?

Closures capture lexical scope, a foundational concept for callbacks and decorators.

Answer:

A closure is created when a nested function captures variables from its enclosing scope. The captured variables are stored in the function's “__closure__” attribute.



31

What are slots (`__slots__`) and why use them? 🤔

Slots save memory by preventing the creation of “`__dict__`” on many instances, crucial in large-scale systems.

Answer:

Defining “`__slots__ = ('x','y')`” on a class restricts its attributes to those names and eliminates the per-instance attribute dict, reducing memory overhead.



Hadil Ben Abdallah
@hadil-ben-abdallah

32

How is Python code compiled and executed internally? 🎬

Understanding the CPython pipeline helps you debug performance and bytecode issues.

Answer:

Source code → **AST** → **bytecode** → **evaluation** by the CPython virtual machine. You can inspect bytecode with the “dis” module.



33

How would you design a scalable web application backend in Python? 🌐

Architectural questions test your ability to apply Python skills to real-world systems.

Answer:

Choose frameworks (FastAPI for async or Django for batteries-included), containerize with Docker, use async or multiprocessing for concurrency, add caching (Redis), and deploy via Kubernetes or serverless platforms.



34

Explain the concept of memoization in Python. 🧠

Memoization caches expensive function results, speeding up repeated calls and dynamic programming.

Answer:

Implement manually with a dict or use “functools.lru_cache” decorator to cache function outputs based on arguments.



35

How does `async/await` syntax work under the hood? 🍪

Grasping `async` internals lets you debug event-loop issues and write more efficient coroutines.

Answer:

“`async def`” functions return a coroutine object. The event loop schedules coroutines, “`await`” suspends execution until the awaited future completes, resuming thereafter.



Hadil Ben Abdallah
@hadil-ben-abdallah

36

**What's the role of the
“__init__.py” file? **

“__init__.py” turns a folder into a package, affecting import behavior and module initialization.

Answer:



An empty or configured “__init__.py” signals Python to treat the directory as a package, you can also expose submodules and set package-level variables there.

37

How does Python's type hinting improve code quality? ✍️

Type hints enable static analysis, better IDE support, and fewer runtime errors in large codebases.

Answer:

Using “typing” annotations (e.g., “def f(x: int) -> str”) lets tools like **mypy** catch type mismatches before runtime and improves documentation.



Hadil Ben Abdallah
@hadil-ben-abdallah

38

How would you create a custom iterator?

Iterators let you build custom loopable objects for domain-specific data streams.

Answer:

Define a class with “`__iter__()`” returning “self” and “`__next__()`” to yield the next value or raise “`StopIteration`”.



39

How does the descriptor protocol (`__get__`, `__set__`, `__delete__`) work? 🔧

Descriptors underpin property, methods, and advanced attribute management in frameworks.

Answer:

A descriptor is an object with any of these methods, when placed in a class, attribute access invokes the protocol:

- “`__get__`” on retrieval
- “`__set__`” on assignment
- “`__delete__`” on deletion



40

Compare Django and FastAPI. Which would you choose for a new project in 2025? 🥊

Framework choice impacts development speed, performance, and scalability of real-world applications.



Answer:

Django offers a full-featured “batteries-included” stack, ORM, admin, and auth, ideal for monolithic apps. **FastAPI** is lightweight, async-first, and faster for APIs. Choose Django for complex business logic and FastAPI for high-performance microservices.



Final Thoughts

Python interviews in 2025 will not just be about syntax, they'll test your **problem-solving, architecture sense, and performance awareness.**

Remember: Always, keep learning, keep building, and you'll stand out in any technical interview!



Found this helpful?



Share to help others



Save to refer to later



Follow Hadil Ben Abdallah for more
amazing content about programming